

Java Exception Handling

Java Backend Interview — Short Notes

1. Exception Hierarchy

- **Throwable** is the root of all exceptions and errors in Java
- **Error** — serious JVM-level problems. Not meant to be caught. e.g., `OutOfMemoryError`, `StackOverflowError`, `VirtualMachineError`
- **Exception** — recoverable conditions. Two branches: checked and unchecked
- **Checked exceptions** — extend `Exception` (not `RuntimeException`). Compiler enforces handling — must catch or declare with `throws`
- **Unchecked exceptions** — extend `RuntimeException`. No compiler enforcement — programmer's responsibility

Class	Type	Must Handle?	Common Examples
Throwable	Root	—	Base class for all exceptions and errors
Error	Unchecked	No	<code>OutOfMemoryError</code> , <code>StackOverflowError</code> , <code>AssertionError</code> , <code>ThreadDeath</code>
Exception	Checked	Yes	<code>IOException</code> , <code>SQLException</code> , <code>ClassNotFoundException</code> , <code>CloneNotSupportedException</code>
RuntimeException	Unchecked	No	<code>NullPointerException</code> , <code>IllegalArgumentException</code> , <code>IndexOutOfBoundsException</code> , <code>ClassCastException</code>
IOException	Checked	Yes	<code>FileNotFoundException</code> , <code>SocketException</code> , <code>EOFException</code>
SQLException	Checked	Yes	Database access errors — connection, query, constraint violations
NullPointerException	Unchecked	No	Accessing member on null reference — most common Java exception
IllegalArgumentException	Unchecked	No	Method receives invalid argument — use to validate inputs
IllegalStateException	Unchecked	No	Object in wrong state for the operation — e.g., iterator exhausted
IndexOutOfBoundsException	Unchecked	No	Array/List index out of range. <code>ArrayIndexOutOfBoundsException</code> , <code>StringIndexOutOfBoundsException</code>
ClassCastException	Unchecked	No	Invalid cast — (String) someObject when not a String
ArithmeticException	Unchecked	No	Arithmetic error — divide by zero is most common
NumberFormatException	Unchecked	No	<code>Integer.parseInt("abc")</code> — invalid string to number
UnsupportedOperationException	Unchecked	No	Operation not supported — <code>Collections.unmodifiableList</code> , <code>List.of()</code> mutations
ConcurrentModificationException	Unchecked	No	Modifying collection while iterating — fail-fast iterator
StackOverflowError	Error	No	Infinite/deep recursion exhausts call stack

OutOfMemoryError	Error	No	JVM heap exhausted — memory leak or too large allocations
------------------	-------	----	---

2. Checked vs Unchecked Exceptions

	Checked Exception	Unchecked Exception
Extends	Exception (but not RuntimeException)	RuntimeException (or Error)
Compiler check	Yes — must catch or declare throws	No — optional to handle
Represents	External failure — IO, DB, network, file system	Programming bug — null, bad cast, bad index, bad arg
Caller expected to	Handle or propagate — recovery is possible	Fix the code — these shouldn't occur in correct programs
Examples	IOException, SQLException, ParseException, InterruptedException	NPE, ClassCastException, IllegalArgumentException, ArithmeticException
Spring/modern	Often wrapped in RuntimeException for cleaner API	Preferred in modern frameworks (Spring uses unchecked)

✓ Modern best practice: use unchecked exceptions for domain/business errors. Checked exceptions for external systems (IO, DB, network) where caller can meaningfully recover

■ Avoid declaring 'throws Exception' or 'throws Throwable' — too broad, forces callers to handle everything

3. try-catch-finally

3.1 Basic Syntax

```
try {
    // code that may throw
    String s = null;
    s.length(); // throws NullPointerException
} catch (NullPointerException e) {
    System.out.println("NPE: " + e.getMessage());
} catch (IllegalArgumentException e) {
    System.out.println("Bad arg: " + e.getMessage());
} finally {
    System.out.println("Always runs – cleanup here");
}
```

3.2 finally Block Rules

- **finally always executes** — whether exception thrown or not, whether caught or not
- Runs even if catch block throws — finally still executes before propagating
- Runs even if there is a return in try or catch — but finally return overrides try/catch return (dangerous!)
- **Only case finally does NOT run:** System.exit() called, JVM crash, or thread killed
- Use for: closing resources, releasing locks, cleanup. But prefer try-with-resources instead

finally return overrides — avoid this pattern:

```
try { return 1; } finally { return 2; } // returns 2 – return in try is suppressed!
```

3.3 Multi-catch (Java 7+)

- Catch multiple exception types in a single catch block — reduces duplication
- Caught exception is implicitly final — cannot reassign inside the block

```
try {
    riskyOperation();
} catch (IOException | SQLException e) { // pipe-separated
    log.error("Operation failed", e);
}
```

3.4 Catch Order — Most Specific First

- More specific (subclass) exceptions must appear before less specific (superclass) ones
- Compiler error if broader catch is listed before narrower — 'Exception already caught'

```
// CORRECT – specific to general
catch (FileNotFoundException e) { ... } // specific
catch (IOException e) { ... } // broader
catch (Exception e) { ... } // broadest last

// WRONG – compiler error
catch (Exception e) { ... } // FileNotFoundException already caught here
catch (FileNotFoundException e) { ... } // unreachable!
```

4. try-with-resources (Java 7+)

- Automatically closes resources — no need for finally block to call close()
- Resource must implement **AutoCloseable** (or Closeable which extends AutoCloseable)
- close() is called in **reverse order** of declaration when try block exits (normally or via exception)
- If both try block AND close() throw, try-block exception is primary; close() exception is **suppressed**

```
// Old way – error-prone
Connection conn = null;
try { conn = getConn(); doWork(conn); }
finally { if(conn!=null) conn.close(); }

// try-with-resources – clean
try (Connection conn = getConnection();
    PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.executeQuery();
} // ps.close() then conn.close() called automatically

// Java 9+: use effectively-final variable declared outside
Connection conn = getConnection();
try (conn) { // just reference – no redeclaration needed
    doWork(conn);
}
```

4.1 Suppressed Exceptions

- If try block throws AND close() also throws, try-block exception propagates; close() exception is attached as suppressed
- Access suppressed via: e.getSuppressed() — returns array of Throwable

```
try {
    throw new RuntimeException("try block");
} // close() also throws → suppressed, attached to RuntimeException
catch (RuntimeException e) {
    Throwable[] suppressed = e.getSuppressed(); // close() exception here
}
```

✓ Always use try-with-resources for anything implementing `AutoCloseable` — streams, connections, readers, writers, channels

4.2 AutoCloseable vs Closeable

Interface	Method	Exception	Notes
AutoCloseable	<code>close()</code>	throws <code>Exception</code>	Java 7. Broader — any exception from <code>close()</code>
Closeable	<code>close()</code>	throws <code>IOException</code>	Java 5. Narrower — only <code>IOException</code> . Extends <code>AutoCloseable</code> . For IO resources

5. throw and throws

5.1 throw — raise an exception

- **throw** is a statement that raises an exception instance
- Can throw any `Throwable` — but throw `Error` or `Throwable` directly only for truly fatal situations
- After throw, execution immediately jumps to nearest matching catch or propagates up the call stack

```
public void setAge(int age) {  
    if (age < 0 || age > 150) {  
        throw new IllegalArgumentException("Invalid age: " + age);  
    }  
    this.age = age;  
}
```

5.2 throws — declare checked exceptions

- **throws** in method signature declares that method may throw specified checked exceptions
- Informs caller they must handle or further declare the exception
- For unchecked exceptions — throws declaration is optional (informational only)
- A method can declare throws for exceptions it doesn't actually throw — but this is confusing

```
public void readFile(String path) throws IOException, FileNotFoundException {  
    // caller must handle IOException or declare throws IOException  
}
```

5.3 Exception Propagation

- If a method doesn't catch an exception, it propagates up the call stack to the caller
- Propagation continues until caught or reaches `main()` — then JVM prints stack trace and terminates thread
- Stack trace shows the full chain of method calls from origin to the uncaught point

```
main() → methodA() → methodB() → methodC() throws IOException  
// If methodC throws and neither methodB nor methodA catch it,  
// IOException propagates to main(). If main() doesn't catch → thread terminates
```

6. Custom Exceptions

- Create custom exceptions for domain-specific errors — meaningful names, proper hierarchy
- **Extend RuntimeException** for unchecked (most common in modern apps)
- **Extend Exception** for checked — when caller must handle and can meaningfully recover
- Always provide: no-arg constructor, message constructor, message+cause constructor, cause constructor
- The **cause** constructor is essential — preserves original exception chain (don't swallow root cause)

```
// Unchecked custom exception (preferred in Spring/modern apps)
```

```

public class UserNotFoundException extends RuntimeException {
    private final Long userId;

    public UserNotFoundException(Long userId) {
        super("User not found with id: " + userId);
        this.userId = userId;
    }

    public UserNotFoundException(Long userId, Throwable cause) {
        super("User not found with id: " + userId, cause);
        this.userId = userId;
    }

    public Long getUserId() { return userId; }
}

// Usage
User user = userRepo.findById(id)
    .orElseThrow(() -> new UserNotFoundException(id));

// Checked custom exception
public class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) { super(message); }
    public InsufficientFundsException(String message, Throwable cause) { super(message, cause); }
}

```

✓ Naming convention: end with 'Exception'. Be specific — OrderNotFoundException, PaymentDeclinedException, InvalidTokenException

■ Don't create one generic AppException for everything — defeats the purpose of typed exceptions

7. Exception Chaining & Stack Traces

7.1 Exception Chaining

- Wrap a low-level exception in a higher-level one to add context — preserves root cause
- Pass original exception as **cause** to new exception constructor
- `getCause()` traverses the chain. Stack trace shows full chain — critical for debugging

```

try {
    // DB layer – low-level
    connection.execute(sql);
} catch (SQLException e) {
    // Wrap with domain exception – don't lose root cause
    throw new OrderPersistenceException("Failed to save order", e); // e = cause
}

// Later: e.getCause() returns the original SQLException
// Stack trace shows both exceptions and where each originated

```

■ Never do: `throw new MyException(e.getMessage())` — this loses the stack trace of the original exception

7.2 Key Throwable Methods

Method	Description
<code>getMessage()</code>	Returns the detail message string set in constructor
<code>getLocalizedMessage()</code>	Like <code>getMessage()</code> — override to provide locale-specific messages
<code>getCause()</code>	Returns the cause Throwable (set via constructor or <code>initCause()</code>)
<code>getClass().getName()</code>	Fully qualified exception class name

<code>toString()</code>	Returns className: message — used in stack trace first line
<code>printStackTrace()</code>	Print full stack trace to stderr. Use <code>log.error(msg, e)</code> in production instead
<code>getStackTrace()</code>	Returns <code>StackTraceElement[]</code> — programmatic access to stack frames
<code>getSuppressed()</code>	Returns suppressed exceptions added via <code>addSuppressed()</code> (try-with-resources)
<code>initCause(cause)</code>	Set cause after construction — only if cause not set in constructor. One-time call
<code>fillInStackTrace()</code>	Captures current stack. Called in constructor. Override to skip (performance)

8. Exception Handling Best Practices

8.1 Do's

- **Catch specific exceptions** — catch `FileNotFoundException` before `IOException`. Never catch `Exception` unless at top level
- **Always preserve cause** — when wrapping: `throw new MyEx("msg", originalException)`. Never swallow cause
- **Log at the right level** — log exception once at the layer that handles it. Don't log-and-throw (double logging)
- **Use try-with-resources** — for any `AutoCloseable` resource
- **Clean up in finally / try-with-resources** — release locks, connections, streams
- **Fail fast** — validate inputs at method start, throw `IllegalArgumentException` early
- **Include context in message** — 'User not found with id: 42' not just 'Not found'
- **Document with @throws Javadoc** — especially for checked exceptions in public API

8.2 Don'ts

- Don't swallow exceptions — empty catch block hides bugs: `catch(Exception e) {}`
- Don't catch `Error` or `Throwable` — you cannot recover from `OutOfMemoryError`
- Don't use exceptions for flow control — they're expensive (stack trace capture) and unreadable
- Don't log and rethrow — causes duplicate log entries. Either log OR rethrow, not both
- Don't declare 'throws Exception' — too broad, forces callers to handle everything
- Don't return null from catch — return `Optional` or throw instead
- Don't ignore `InterruptedException` — always restore interrupt flag: `Thread.currentThread().interrupt()`

8.3 InterruptedException — Special Handling

- `InterruptedException` signals that the thread was interrupted — clearing the interrupt flag
- If you catch `InterruptedException` and don't rethrow it, you MUST restore the interrupt status

```
// WRONG – swallows interrupt signal
try { Thread.sleep(1000); } catch (InterruptedException e) { /* ignore */ }

// CORRECT option 1: rethrow
try { Thread.sleep(1000); } catch (InterruptedException e) { throw e; }

// CORRECT option 2: restore interrupt flag if can't rethrow
try { Thread.sleep(1000); }
catch (InterruptedException e) {
    Thread.currentThread().interrupt(); // restore flag
    // handle graceful shutdown
}
```

9. Exceptions in Java 8+ Streams & Lambdas

- Functional interfaces (`Predicate`, `Function`, `Consumer`) do not declare checked exceptions

- Checked exceptions inside lambdas must be caught or wrapped — compiler enforces this

```
// Problem: Files.readAllBytes throws IOException (checked)
List paths = List.of("/a.txt", "/b.txt");
// paths.stream().map(Files::readAllBytes) // COMPILER ERROR – IOException not handled

// Solution 1: wrap in try-catch inside lambda
paths.stream().map(p -> {
    try { return Files.readAllBytes(Path.of(p)); }
    catch (IOException e) { throw new RuntimeException(e); } // wrap
});

// Solution 2: extract to a helper that wraps the exception
@FunctionalInterface
interface ThrowingFunction {
    R apply(T t) throws Exception;
    static Function wrap(ThrowingFunction fn) {
        return t -> { try { return fn.apply(t); }
            catch (Exception e) { throw new RuntimeException(e); } };
    }
}

paths.stream().map(ThrowingFunction.wrap(Files::readAllBytes)); // clean
```

9.1 CompletableFuture Exception Handling

- Exceptions inside supplyAsync/thenApply are wrapped in CompletionException
- exceptionally(fn) — catches exception, returns fallback value
- handle((result, ex) -> ...) — handles both success and failure in one step

```
CompletableFuture.supplyAsync(() -> fetchData())
    .exceptionally(ex -> { log.error("Failed", ex); return defaultValue; })
    .thenAccept(result -> process(result));

// handle() – access both result and exception
.handle((result, ex) -> {
    if (ex != null) return fallback;
    return result;
});
```

10. Exception Handling in Spring Boot

10.1 @ExceptionHandler

- Handle exceptions within a specific controller — method-level or controller-level
- Return ResponseEntity with appropriate HTTP status and error body

```
@RestController
public class UserController {
    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity handleNotFound(UserNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse(ex.getMessage()));
    }
}
```

10.2 @ControllerAdvice / @RestControllerAdvice (Global Handler)

- Global exception handling across ALL controllers — single place for all exception mapping

- `@RestControllerAdvice` = `@ControllerAdvice` + `@ResponseBody`
- Methods annotated with `@ExceptionHandler` inside — applied globally

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(UserNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleNotFound(UserNotFoundException ex) {
        return new ErrorResponse("NOT_FOUND", ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponse handleValidation(MethodArgumentNotValidException ex) {
        String errors = ex.getBindingResult().getFieldErrors().stream()
            .map(e -> e.getField()+" : "+e.getDefaultMessage())
            .collect(Collectors.joining(", "));
        return new ErrorResponse("VALIDATION_ERROR", errors);
    }

    @ExceptionHandler(Exception.class) // fallback for all others
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public ErrorResponse handleGeneric(Exception ex) {
        log.error("Unexpected error", ex);
        return new ErrorResponse("INTERNAL_ERROR", "Something went wrong");
    }
}
```

10.3 Standard Error Response Structure

```
// Consistent error response DTO
public record ErrorResponse(
    String code,
    String message,
    LocalDateTime timestamp,
    String path
) {}
```

10.4 HTTP Status Mapping Convention

Exception Type	HTTP Status	Spring Example
Resource not found	404 NOT_FOUND	UserNotFoundException, OrderNotFoundException
Validation failure	400 BAD_REQUEST	MethodArgumentNotValidException, ConstraintViolationException
Invalid input / bad argument	400 BAD_REQUEST	IllegalArgumentException, HttpMessageNotReadableException
Unauthorized — not logged in	401 UNAUTHORIZED	AuthenticationException, ExpiredJwtException
Forbidden — no permission	403 FORBIDDEN	AccessDeniedException
Conflict — duplicate/state	409 CONFLICT	DataIntegrityViolationException, OptimisticLockingFailureException
Unprocessable / business rule	422 UNPROCESSABLE_ENTITY	BusinessRuleViolationException, InsufficientFundsException

External service failure	502 BAD_GATEWAY / 503 UNAVAILABLE	Downstream API timeout, circuit open
Unexpected / unhandled	500 INTERNAL_SERVER_ERROR	Catch-all Exception handler

11. Important Patterns

11.1 Exception Translation

- Translate low-level/infrastructure exceptions to domain exceptions at layer boundaries
- Repository layer: catch `SQLException` → throw `OrderNotFoundException` or `DataAccessException`
- This decouples upper layers from infrastructure details

```
// Repository
public Order findById(Long id) {
    try {
        return jdbcTemplate.queryForObject(SQL, mapper, id);
    } catch (EmptyResultDataAccessException e) {
        throw new OrderNotFoundException(id, e); // translate + preserve cause
    }
}
```

11.2 Null Safety — Avoiding `NullPointerException`

- Use **Optional** for values that may be absent — forces caller to handle absence
- Validate method arguments early with `Objects.requireNonNull()` or Guava Preconditions
- Java 14+ **Helpful `NullPointerException`s** — JVM tells you exactly which variable was null

```
// Optional instead of returning null
Optional findUser(Long id) { return Optional.ofNullable(repo.find(id)); }
user.orElseThrow(() -> new UserNotFoundException(id));

// Validate inputs
public void process(String name) {
    Objects.requireNonNull(name, "name must not be null");
    if (name.isBlank()) throw new IllegalArgumentException("name must not be blank");
}
```

11.3 Rethrowing Exceptions

```
// Rethrow same exception – preserves type
catch (IOException e) { throw e; }

// Rethrow as unchecked – unwrap for cleaner API
catch (Exception e) { throw new RuntimeException("Unexpected error", e); }

// Java 8+ sneaky throw pattern (Lombok @SneakyThrows does this)
@SneakyThrows // Lombok – rethrows checked as unchecked without wrapping
public void read() { Files.readAllBytes(path); } // IOException thrown transparently
```

12. Frequently Asked & Tricky Interview Questions

Hierarchy & Basics

Q1. What is the difference between Error and Exception?

→ Exception: recoverable conditions — checked (`IOException`) or unchecked (`NPE`). Catch and handle. Error: severe JVM-level failures (`OutOfMemoryError`, `StackOverflowError`) — not meant to be caught. Errors indicate unrecoverable

situations

Q2. What is the difference between checked and unchecked exceptions?

→ Checked: extend `Exception` (not `RuntimeException`). Compiler enforces handling — must catch or declare throws. Represent external failures (IO, DB). Unchecked: extend `RuntimeException`. No compiler enforcement. Represent programming bugs. Modern apps prefer unchecked for domain errors

Q3. Can we catch `Error` in Java?

→ Syntactically yes — but you shouldn't. Errors indicate JVM is in an unrecoverable state. Catching `OutOfMemoryError` won't help — heap is still exhausted. Exception: sometimes catch `StackOverflowError` to return a graceful response, but this is rare

Q4. What is `RuntimeException`? Give five examples.

→ Unchecked exception subclass — no compiler-enforced handling. Examples: `NullPointerException` (null dereference), `ClassCastException` (invalid cast), `IllegalArgumentException` (bad input), `ArrayIndexOutOfBoundsException` (bad index), `NumberFormatException` (invalid string-to-number)

Q5. Is it mandatory to handle `RuntimeException`?

→ No — compiler does not require it. But you should handle where it makes sense (e.g., parse user input). If unhandled, it propagates up and may terminate the thread with a stack trace

try-catch-finally

Q6. Does finally always execute?

→ Almost always — yes. Exceptions: `System.exit()` call, JVM crash, thread kill (`stop()`), infinite loop in try. It runs even if catch block throws, even if try/catch has return statement — though finally-return overrides try-return (avoid this pattern)

Q7. What happens when both catch and finally throw exceptions?

→ The finally exception propagates — the catch exception is lost (suppressed). This is why you should not throw from finally. If using try-with-resources, `close()` exceptions are suppressed and attached to the try-block exception via `getSuppressed()`

Q8. Can we have try without catch?

→ Yes — try-finally is valid: `try { } finally { }`. Also try-with-resources without catch is valid. But bare `try { }` without catch or finally is a compile error

Q9. What is the order of catch blocks? Why does it matter?

→ Most specific (subclass) first, least specific (superclass) last. If superclass appears first, all subclass exceptions match it — subclass catch blocks become unreachable. Compiler gives error: 'exception already caught'

Q10. What is multi-catch? Any restrictions?

→ Java 7+: `catch (IOException | SQLException e)` — catches multiple types in one block. The variable `e` is implicitly final — cannot reassign. Exception types in multi-catch must not be in subclass-superclass relationship (compiler error if so)

try-with-resources

Q11. What is try-with-resources? What interface must resources implement?

→ Java 7+ feature — automatically calls `close()` on resources when block exits (normally or via exception). Resource must implement `AutoCloseable` (`close()` throws `Exception`) or `Closeable` (`close()` throws `IOException`)

Q12. What are suppressed exceptions in try-with-resources?

→ If both try block and `close()` throw, the try-block exception propagates as primary. The `close()` exception is suppressed — attached to primary via `addSuppressed()`. Retrieve via `e.getSuppressed()`. Before Java 7, `close()` exception would silently replace the try-block exception

Q13. Can we declare multiple resources in try-with-resources?

→ Yes — separated by semicolons. They are closed in reverse order of declaration. If `conn` and `ps` both declared, `ps.close()` is called first, then `conn.close()`

throw, throws, custom

Q14. Difference between throw and throws?

→ `throw`: statement that raises an exception instance (`throw new SomeException()`). `throws`: keyword in method signature declaring that method may throw checked exceptions (forces callers to handle). `throw` is imperative; `throws` is declarative

Q15. Can we throw checked exceptions from unchecked exception handlers?

→ Yes syntactically, but the checked exception must still be declared in throws or caught. You can re-wrap: throw new RuntimeException(exception) to avoid declaring throws

Q16. How do you create a custom exception? What constructors should it have?

→ Extend RuntimeException (unchecked) or Exception (checked). Provide: no-arg, message-only, message+cause, cause-only constructors. The cause constructor is critical — preserves exception chain. Add relevant fields (e.g., userId, orderId) for richer context

Q17. When would you use a checked vs unchecked custom exception?

→ Checked: when caller can meaningfully recover and must be forced to handle it (e.g., InsufficientFundsException in banking — caller must handle). Unchecked: when it represents a programming error or recovery isn't expected (e.g., UserNotFoundException — let global handler convert to 404)

Best Practices

Q18. What is exception swallowing? Why is it bad?

→ Empty catch block: catch(Exception e) { } — silently ignores exception. The error appears to succeed but behavior is undefined. Bugs become extremely hard to find. Always at minimum: log the exception

Q19. What is log-and-throw antipattern?

→ Catching exception, logging it, then rethrowing — causes duplicate log entries as exception propagates up. Either log it (handle it here) or rethrow it (let upper layer handle), not both

Q20. How do you handle InterruptedException correctly?

→ Option 1: rethrow it (propagate interrupt signal up). Option 2: if you can't rethrow, restore interrupt status: Thread.currentThread().interrupt() — so caller knows thread was interrupted. Never silently swallow InterruptedException

Q21. What is exception chaining? Why is it important?

→ Wrapping low-level exception in higher-level: throw new DomainException('msg', originalException). Preserves root cause — getCause() and stack trace show full chain. Without chaining, root cause is lost and debugging is very hard

Q22. Should you catch Exception or Throwable in a global handler?

→ Catch Exception — not Throwable. Catching Throwable also catches Errors (OutOfMemoryError, etc.) which you cannot meaningfully handle. Catch Exception in global handler as the broadest safety net for unexpected application errors

Java 8+ & Spring

Q23. Why can't you use checked exceptions inside lambdas?

→ Standard functional interfaces (Function, Consumer, Predicate) don't declare throws — checked exceptions must be caught inside the lambda or wrapped. Solutions: try-catch inside lambda (verbose), or create a wrapper ThrowingFunction that wraps to RuntimeException

Q24. How does CompletableFuture handle exceptions?

→ Exceptions inside async stages are wrapped in CompletionException. exceptionally(fn): catches and provides fallback value. handle((result,ex)->...): handles both result and exception. If you call join() and task failed, CompletionException is thrown — getCause() gets original

Q25. How does Spring Boot handle exceptions globally?

→ @RestControllerAdvice with @ExceptionHandler methods — maps exception types to HTTP responses. Applied across all controllers. Use @ResponseStatus to set HTTP code. The most specific @ExceptionHandler wins. Fallback: catch Exception for 500 responses

Q26. What HTTP status should UserNotFoundException return?

→ 404 NOT_FOUND — resource not found. Map in @RestControllerAdvice: @ExceptionHandler(UserNotFoundException.class) + @ResponseStatus(NOT_FOUND). Return consistent error body: {code, message, timestamp, path}

Q27. What is ControllerAdvice vs RestControllerAdvice?

→ @ControllerAdvice: handles exceptions and returns ModelAndView (for MVC). @RestControllerAdvice = @ControllerAdvice + @ResponseBody — returns JSON/XML bodies. Use @RestControllerAdvice for REST APIs

Q28. What is the difference between @ResponseStatus on exception class vs on handler method?

→ On exception class: `@ResponseStatus(HttpStatus.NOT_FOUND)` — Spring MVC automatically maps that exception to that HTTP status. On handler method: `@ResponseStatus` decorates the handler method. Handler method overrides the annotation on exception class

Tricky & Advanced

Q29. What happens to exceptions thrown in a static initializer?

→ Wrapped in `ExceptionInInitializerError` (an `Error`). The class fails to load. Subsequent attempts to use the class throw `NoClassDefFoundError`. Very hard to debug — be careful with code in static {}

Q30. Can a constructor throw an exception?

→ Yes — any exception. If constructor throws, object is not created. For checked exceptions, the constructor must declare throws or callers must catch. Common for validation: throw `IllegalArgumentException` if args invalid

Q31. What is the difference between `ClassNotFoundException` and `NoClassDefFoundError`?

→ `ClassNotFoundException`: checked exception — thrown when `Class.forName()` or `ClassLoader.loadClass()` cannot find the class at runtime. `NoClassDefFoundError`: error — class was available at compile time but not at runtime (missing from classpath, failed static initializer)

Q32. What is `StackOverflowError`? How to avoid?

→ Error thrown when call stack exceeds JVM limit — infinite recursion most common cause. Fix: add proper base case to recursion, or convert to iterative, or increase stack size with `-Xss` JVM flag (rarely the right fix)

Q33. Can finally block prevent exception propagation?

→ Yes — if finally block has a return statement or throws a different exception, the original exception is suppressed. This is a dangerous antipattern. Avoid return and throw in finally blocks

Q34. What is Lombok `@SneakyThrows`?

→ Annotation that lets you throw checked exceptions without declaring them — uses bytecode trick to rethrow as-is without wrapping. Useful for lambdas and clean APIs. Controversial — callers don't know exception can be thrown. Use carefully

Q35. How do you test exception throwing with JUnit 5?

→ `assertThrows(ExceptionType.class, () -> methodThatThrows())`. Returns the exception — can assert on message: `assertEquals('expected msg', ex.getMessage())`. `assertDoesNotThrow()` for negative test. Old JUnit 4 used `@Test(expected=SomeException.class)`

Q36. What is `Optional` and how does it help with `NullPointerException`?

→ `Optional` wraps a value that may or may not be present — forces explicit absence handling. `orElse(default)`, `orElseGet(supplier)`, `orElseThrow(exceptionSupplier)`. Eliminates null returns that cause NPE downstream. Java 8+

Q37. What are Java 14 Helpful `NullPointerException`s?

→ JVM now says exactly which variable was null: 'Cannot invoke `String.length()` because `user.address` is null' instead of just NPE at line X. Enable with `-XX:+ShowCodeDetailsInExceptionMessages` (default on in Java 17+)

Q38. How do you handle exceptions in Spring `@Async` methods?

→ Exceptions in `@Async` methods are not propagated to caller — they're lost by default. Solution: configure `AsyncUncaughtExceptionHandler` via `AsyncConfigurer`, or return `CompletableFuture` and handle with `exceptionally/handle`

Q39. What is the difference between `Exception.printStackTrace()` and using a logger?

→ `printStackTrace()` writes to `stderr` — no control over format, cannot be filtered, not sent to log aggregators. `logger.error('msg', e)` uses logging framework — structured, configurable level/format, sent to log aggregation systems (ELK, Splunk). Always use logger in production

Q40. How would you design exception handling for a REST microservice?

→ 1) Domain-specific unchecked exceptions (`UserNotFoundException`, `OrderException`). 2) `@RestControllerAdvice` global handler mapping exceptions to HTTP statuses. 3) Consistent `ErrorResponse` DTO {code, message, timestamp, traceId}. 4) Log with traceId for correlation. 5) Never expose stack traces to client. 6) Return 500 for unexpected errors with generic message

Quick Fire

Q41. Is it possible to rethrow the same exception?

→ Yes: `catch (IOException e) { throw e; }` — preserves original stack trace. Java 7+ allows this for more specific exception types even if method declares broader throws

Q42. What exception does `Integer.parseInt('abc')` throw?

→ `NumberFormatException` (unchecked `RuntimeException`) — invalid string for numeric parsing

Q43. What exception does `arr[10]` throw when `arr.length` is 5?

→ `ArrayIndexOutOfBoundsException`: Index 10 out of bounds for length 5 — unchecked

Q44. What exception does `(String) new Integer(5)` throw?

→ `ClassCastException`: Integer cannot be cast to String — unchecked

Q45. What exception does `5/0` throw in Java?

→ `ArithmeticException`: / by zero — unchecked. Note: `5.0/0.0` returns `Double.POSITIVE_INFINITY` — no exception

Q46. Can we have try-catch inside a finally block?

→ Yes — perfectly valid. Useful if cleanup code (`close()`) might throw and you want to handle it without losing the original exception

Q47. What is `ExceptionInInitializerError`?

→ Error thrown when static initializer block throws an unchecked exception. The class cannot be loaded. Subsequent access throws `NoClassDefFoundError`

Q48. What does `getCause()` return for a top-level exception?

→ null — there is no cause if exception was created without one. Always check for null before calling `getCause().getMessage()`

Q49. Can we catch multiple exceptions in the same catch block without multi-catch?

→ No — before Java 7, you needed separate catch blocks for each. Java 7+ multi-catch: `catch (Ex1 | Ex2 e) {}` — but the types cannot be in inheritance relationship

Q50. What is the difference between `final`, `finally`, and `finalize`?

→ `final`: keyword — final class (no subclass), final method (no override), final variable (no reassign). `finally`: block in try-catch that always executes — cleanup. `finalize()`: deprecated method on `Object` called by GC before collecting — unreliable, avoid